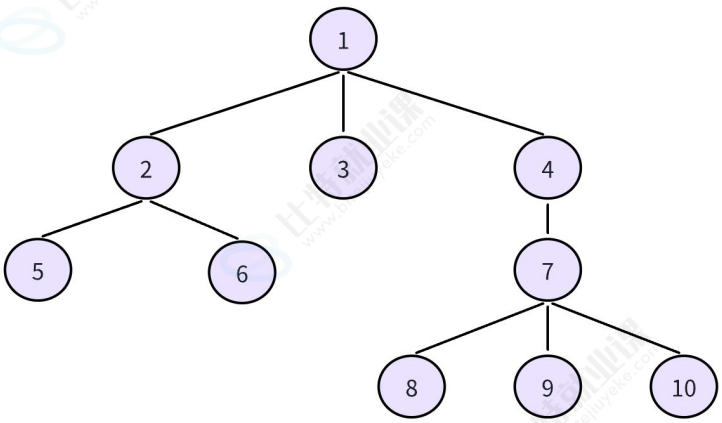
3. 并查集

3.1 双亲表示法

接下来要学习到的并查集，本质上就是用双亲表示法实现的森林。因此，我们先认识一下双亲表示法。

在学习树这个数据结构的时，讲到树的存储方式有很多种：孩子表示法，双亲表示法、孩子双亲表示法以及孩子兄弟表示法等。对一棵树而言，除了根节点外，其余每个结点一定有且仅有一个双亲，双亲表示法就是根据这个特点存储树的，也就是把每个结点的双亲存下来。因此，我们可以采用数组来存储每个结点的父亲结点的编号，这就实现了双亲表示法(so easy)。

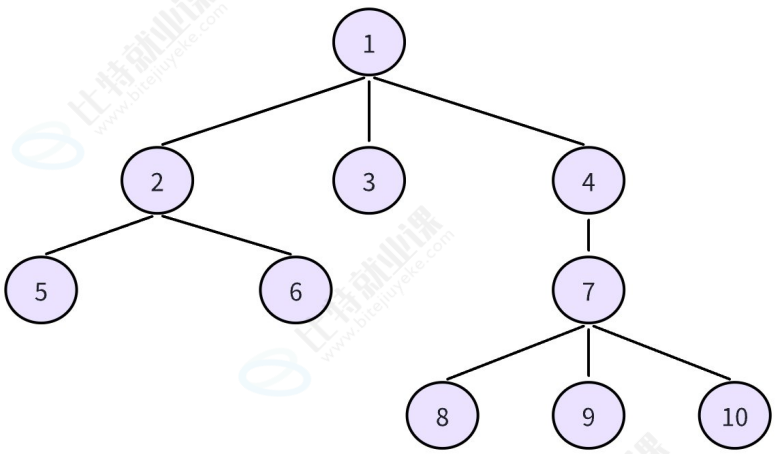


树

下标	fa
1	-1
2	1
3	1
4	1
5	2
6	2
7	4
8	7
9	7
10	7

树的双亲表示

但是，在实现并查集的时，我们一般让**根节点自己指向自己**。因此，上述存储就变成：



树

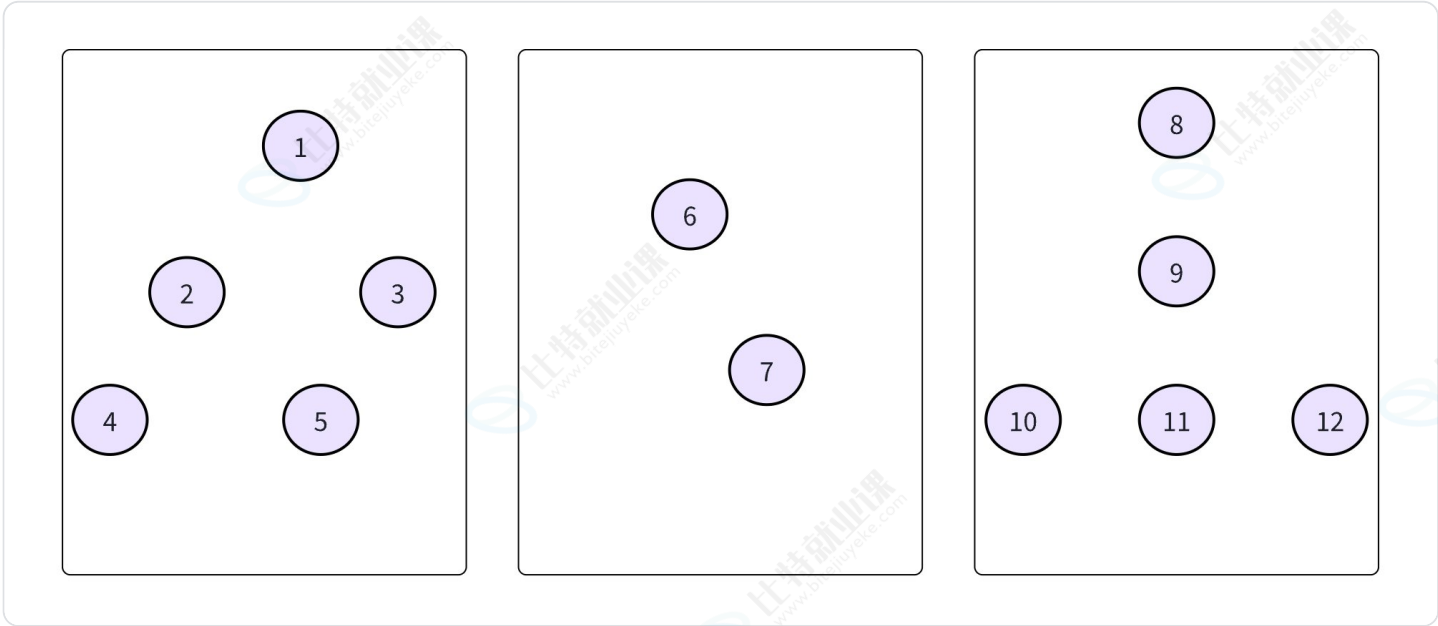
下标	fa
1	1
2	1
3	1
4	1
5	2
6	2
7	4
8	7
9	7
10	7

树的双亲表示

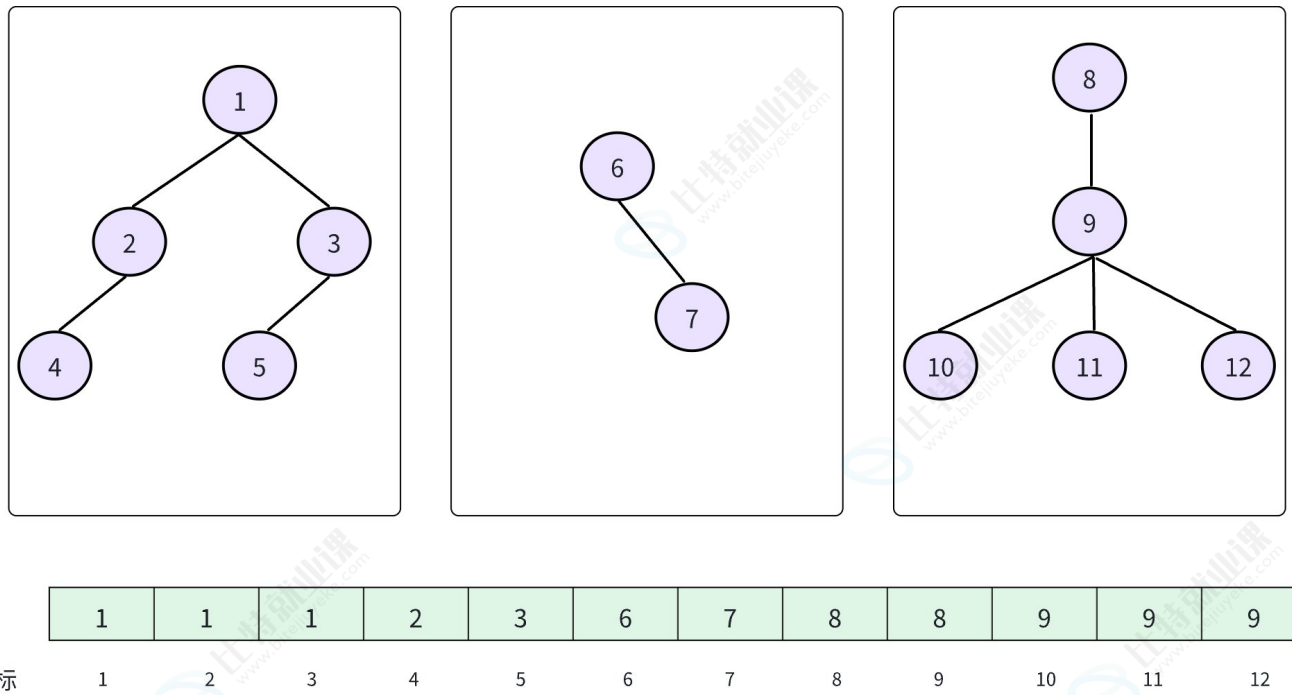
3.2 并查集的概念

在有些问题中，我们需要维护若干个集合，并且基于这些集合要频繁执行下面的操作：

- **查询操作**：查找元素 x 属于哪一个集合。一般会在每个集合中选取一个元素作为代表，查询的是这个集合中的代表元素；
- **合并操作**：将元素 x 所在的集合与元素 y 所在的集合合并成一个集合；（注意，合并的是元素所在的集合，不是这两个元素！）
- **判断操作**：判断元素 x 和 y 是否在同一个集合。



并查集（Union Find）：是一种用于维护元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素，根节点来代表整个集合。



fa	1	1	1	2	3	6	7	8	8	9	9	9
下标	1	2	3	4	5	6	7	8	9	10	11	12

3.3 并查集的实现

3.3.1 初始化

> 初始状态下，所有的元素单独成为一个集合：
>
> - 让元素自己指向自己即可。

代码实现：

```
const int N = 1e6 + 10;
int n;
int fa[N]; // 双亲表示法所需的数组

// 初始化并查集
void init()
{
    for(int i = 1; i <= n; i++) fa[i] = i;
}
```

3.3.2 查询操作

> 查询操作是并查集的核心操作，其余所有的操作都是基于查询操作实现的！
>
> 找到元素 x 所属的集合：
>
> - 一直向上找爸爸~

代码实现：

```
// 查询操作
int find(int x)
{
    if(fa[x] == x) return x;
    return find(fa[x]);

    // 一行实现
    return fa[x] == x ? x : find(fa[x]);
}
```

3.3.3 合并操作

> 将元素 x 所在的集合与元素 y 所在的集合合并成一个集合：
>
> - 让元素 x 所在树的根节点指向元素 y 所在树的根节点。
>
> （反过来也是可以的）

代码实现：

```
// 合并操作
void un(int x, int y) // 注意，函数名字不能用 union，因为它是 C++ 的关键字
{
    int fx = find(x);
    int fy = find(y);
    fa[fx] = fy;
}
```

3.3.4 判断操作

> 判断元素 x 和元素 y 是否在同一集合：

- 看看两者所在树的根节点是否相同。

代码实现：

```
// 判断是否在同一集合
bool issame(int x, int y)
{
    return find(x) == find(y);
}
```

3.4 并查集的优化

极端情况：在合并的过程中，整棵树变成一个链表。

路径压缩：在查询时，把被查询的节点到根节点的路径上的所有节点的父节点设置为根节点，从而减小树的深度。也就是说，在向上查询的同时，把在路径上的每个节点都直接连接到根上，以后查询时就能直接查询到根节点。

代码实现：

```
// 找根节点 - 路径压缩
int find(int x)
{
    if(fa[x] == x) return x;
    return fa[x] = find(fa[x]);

    // 一行实现
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}
```

还有一种优化方式是按秩合并，但是基本上不用按秩合并，并查集的时间复杂度就很优秀了。感兴趣的同学可以搜一下按秩合并，按照大家现在的水平，应该很容易就能看懂~

在《算法导论》中有严格的证明，并查集查询根节点的最坏时间复杂度为 $O(\alpha(n))$，是一个很小的常数。因此，并查集查询以及合并的效率近似可以看成 $O(1)$。

3.5 普通并查集

3.5.1 【模板】并查集

> **题目来源：** 洛谷
>
> **题目链接：** P3367 【模板】并查集
>
> **难度系数：** ★★

【题目描述】

> 如题，现在有一个并查集，你需要完成合并和查询操作。

【输入描述】

> 第一行包含两个整数 N, M，表示共有 N 个元素和 M 个操作。
>
> 接下来 M 行，每行包含三个整数 Z_i, X_i, Y_i。
>
> 当 $Z_i = 1$ 时，将 X_i 与 Y_i 所在的集合合并。
>
> 当 $Z_i = 2$ 时，输出 X_i 与 Y_i 是否在同一集合内，是的输出 `Y`；否则输出 `N`。

【输出描述】

> 对于每一个 $Z_i = 2$ 的操作，都有一行输出，每行包含一个大写字母，为 `Y` 或者 `N`。

【示例一】

> 输入：
>
> ```
> 4 7
> 2 1 2
> 1 1 2
> 2 1 2
> 1 3 4
> 2 1 4
> 1 2 3
> 2 1 4
> ```
>
> 输出：
>
> ```
> N
> Y
> N
> Y
> ```

【解法】

模板题~

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 2e5 + 10;

int n;
int fa[N];

int find(int x)
{
    if(fa[x] == x) return x;
    return fa[x] = find(fa[x]);
}

int main()
{
    int T;
    cin >> n >> T;

    // 初始化
    for(int i = 1; i <= n; i++) fa[i] = i;

    while(T--)
    {
        int z, x, y; cin >> z >> x >> y;
        if(z == 1) // 合并
        {
            int fx = find(x);
            int fy = find(y);
            fa[fx] = fy;
        }
        else // 判断
        {
            if(find(x) == find(y)) cout << "Y" << endl;
            else cout << "N" << endl;
        }
    }

    return 0;
}
```

3.5.2 亲戚

> **题目来源：** 洛谷
>
> **题目链接：** P1551 亲戚
>
> **难度系数：** ★★

【题目描述】

> 规定：x 和 y 是亲戚，y 和 z 是亲戚，那么 x 和 z 也是亲戚。如果 x，y 是亲戚，那么 x 的亲戚都是 y 的亲戚，y 的亲戚也都是 x 的亲戚。

【输入描述】

> 第一行：三个整数 n,m,p，$(n,m,p \leq 5000)$，分别表示有 n 个人，m 个亲戚关系，询问 p 对亲戚关系。
>
> 以下 m 行：每行两个数 $M_i,M_j,1 \leq M_i,M_j \leq n$，表示 M_i 和 M_j 具有亲戚关系。
>
> 接下来 p 行：每行两个数 P_i,P_j，询问 P_i 和 P_j 是否具有亲戚关系。

【输出描述】

> p 行，每行一个 `Yes` 或 `No`。表示第 i 个询问的答案为“具有”或“不具有”亲戚关系。

【示例一】

> 输入：
>
> ```
> 6 5 3
> 1 2
> 1 5
> 3 4
> 5 2
> 1 3
> 1 4
> 2 3
> 5 6
> ```
>
> 输出：
>
> ```
> Yes
> Yes
> No
> ```

【解法】

具有亲戚关系的两个集合就合并在一个集合中。因此，可以用并查集解决。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 5010;

int n, m, p;
int fa[N];

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

void un(int x, int y)
{
    int fx = find(x), fy = find(y);
    fa[fy] = fx;
}

bool issame(int x, int y)
{
    return find(x) == find(y);
}

int main()
{
    cin >> n >> m >> p;

    // 初始化
    for(int i = 1; i <= n; i++) fa[i] = i;

    while(m--)
    {
        int x, y; cin >> x >> y;
        un(x, y);
    }

    while(p--)
```

```
40      {
41          int x, y; cin >> x >> y;
42          if(issame(x, y)) cout << "Yes\n";
43          else cout << "No\n";
44      }
45
46      return 0;
47  }
```

3.5.3 Lake Counting

题目来源： 洛谷

题目链接： P1596 [USACO10OCT] Lake Counting S

难度系数： ★★

【题目描述】

由于近期的降雨，雨水汇集在农民约翰的田地不同的地方。我们用一个 $N \times M (1 \leq N, M \leq 100)$ 的网格图表示。每个网格中有水（`W`） 或是旱地（`.`）。一个网格与其周围的八个网格相连，而一组相连的网格视为一个水坑。约翰想弄清楚他的田地已经形成了多少水坑。给出约翰田地的示意图，确定当中有多少水坑。

【输入描述】

输入第 1 行：两个空格隔开的整数： N 和 M。

第 2 行到第 $N+1$ 行：每行 M 个字符，每个字符是 `W` 或 `.`，它们表示网格图中的一排。字符之间没有空格。

【输出描述】

输出一行，表示水坑的数量。

【示例一】

输入：

```
10 12
W........WW.
.WWW.....WWW
....WW...WW.
.........WW.
.........W..
..W......W..
```

> .W.W.....WW.
>
> W.W.W.....W.
>
> .W.W.....W.
>
> ..W.....W.
>
> 输出：
>
> 3

【解法】

遍历整个矩阵，每次遇到一个水坑时，就把这个水坑右、下，左下以及右下的水坑合并在一起。

最终判断一下一共有多少个集合。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110;

int n, m;
char a[N][N];
int fa[N * N];

int dx[] = {0, 1, 1, 1};
int dy[] = {1, 1, 0, -1};

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

void un(int x, int y)
{
    fa[find(x)] = find(y);
}

int main()
{
    cin >> n >> m;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
```

```
            cin >> a[i][j];

    // 初始化
    for(int i = 0; i < n * m; i++) fa[i] = i;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(a[i][j] == '.') continue;

            for(int k = 0; k < 4; k++)
            {
                int x = i + dx[k], y = j + dy[k];
                if(y >= 0 && a[x][y] == 'W')
                {
                    un(i * m + j, x * m + y);
                }
            }
        }
    }

    int ret = 0;
    for(int i = 0; i < n * m; i++)
    {
        // 一维转二维
        int x = i / m, y = i % m;
        if(a[x][y] == 'W' && fa[i] == i) ret++;
    }

    cout << ret << endl;

    return 0;
}
```

3.5.4 程序自动分析

> **题目来源：** 洛谷
>
> **题目链接：** P1955 [NOI2015] 程序自动分析
>
> **难度系数：** ★★★

【题目描述】

> 在实现程序自动分析的过程中，常常需要判定一些约束条件是否能被同时满足。

考虑一个约束满足问题的简化版本：假设 $x_1, x_2, x_3, \cdots$ 代表程序中出现的变量，给定 n 个形如 $x_i = x_j$ 或 $x_i \neq x_j$ 的变量相等/不等的约束条件，请判定是否可以分别为每一个变量赋予恰当的值，使得上述所有约束条件同时被满足。例如，一个问题中的约束条件为：$x_1 = x_2, x_2 = x_3, x_3 = x_4, x_4 \neq x_1$，这些约束条件显然是不可能同时被满足的，因此这个问题应判定为不可被满足。

现在给出一些约束满足问题，请分别对它们进行判定。

【输入描述】

输入的第一行包含一个正整数 t ，表示需要判定的问题个数。注意这些问题之间是相互独立的。

对于每个问题，包含若干行：

第一行包含一个正整数 n ，表示该问题中需要被满足的约束条件个数。接下来 n 行，每行包括三个整数 i, j, e，描述一个相等/不等的约束条件，相邻整数之间用单个空格隔开。若 $e = 1$，则该约束条件为 $x_i = x_j$。若 $e = 0$，则该约束条件为 $x_i \neq x_j$。

【输出描述】

输出包括 t 行。

输出文件的第 k 行输出一个字符串 `YES` 或者 `NO`（字母全部大写），`YES` 表示输入中的第 k 个问题判定为可以被满足，`NO` 表示不可被满足。

【示例一】

输入：

```
2
2
1 2 1
1 2 0
2
1 2 1
2 1 1
```

输出：

```
NO
YES
```

【解法】

先利用并查集维护所有相等的信息，然后遍历所有的不相等信息，判断一下是否合法。

因为数据范围的问题，需要先对所有的数离散化处理。

【参考代码】

```cpp
#include <iostream>
#include <unordered_map>
#include <algorithm>

using namespace std;

const int N = 1e5 + 10;

int n;
struct node
{
    int x, y, e;
}a[N];

// 离散化
int pos;
int disc[N * 2];
unordered_map<int, int> mp;

// 并查集
int fa[N * 2];

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

void un(int x, int y)
{
    fa[find(x)] = find(y);
}

bool issame(int x, int y)
{
    return find(x) == find(y);
}

bool solve()
{
    cin >> n;
    // 清空数据
    pos = 0;
    mp.clear();

```

```cpp
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i].x >> a[i].y >> a[i].e;
        disc[++pos] = a[i].x; disc[++pos] = a[i].y;
    }

    // 离散化
    sort(disc + 1, disc + 1 + pos);
    int cnt = 0;
    for(int i = 1; i <= pos; i++)
    {
        int x = disc[i];
        if(mp.count(x)) continue;

        cnt++;
        mp[x] = cnt;
    }

    // 初始化
    for(int i = 1; i <= cnt; i++) fa[i] = i;

    // 1. 把所有相等的信息，用并查集维护起来
    for(int i = 1; i <= n; i++)
    {
        int x = a[i].x, y = a[i].y, e = a[i].e;
        if(e == 1) un(mp[x], mp[y]);
    }

    // 2. 拿出不等的信息，判断是否合法
    for(int i = 1; i <= n; i++)
    {
        int x = a[i].x, y = a[i].y, e = a[i].e;
        if(e == 0)
        {
            if(issame(mp[x], mp[y])) return false;
        }
    }

    return true;
}

int main()
{
    int T; cin >> T;

    while(T--)
    {
```

```
92          if(solve()) cout << "YES" << endl;
93          else cout << "NO" << endl;
94      }
95
96      return 0;
97  }
```

3.6 扩展域并查集

普通的并查集只能解决各元素之间仅存在一种相互关系，比如《亲戚》题目中：

- a 和 b 是亲戚关系，b 和 c 是亲戚关系，这时就可以查找出 a 和 c 也存在亲戚关系。

但如果存在各元素之间存在多种相互关系，普通并查集就无法解决。比如下面的案例：

- a 和 b 是敌人关系，b 和 c 是敌人关系，但是 a 和 c 其实不是敌人关系，而是另一种朋友关系。

此时，就不仅仅是简单的敌人关系，还是出现一种朋友关系。

解决这类问题就需要对并查集进行扩展：将每个元素拆分成多个域，每个域代表一种状态或者关系。通过维护这些域之间的关系，来处理复杂的约束条件。

敌人朋友问题中，我们会将 x 分成两个域，朋友域 x 以及敌人域 y：

- x 和 y 是朋友，正常处理，把 x 和 y 合并成一个集合；
- x 和 y 是敌人：那么 x 和 y 的敌人 $y+n$ 就是朋友，合并 x 与 $y+n$；y 和 x 的敌人 $x+n$ 就是朋友，合并 y 与 $x+n$。

这样就可以利用两个域，将所有的关系维护起来。

3.6.1 团伙

> **题目来源：** 洛谷
>
> **题目链接：** P1892 [BOI2003] 团伙
>
> **难度系数：** ★★★

【题目描述】

> 现在有 n 个人，他们之间有两种关系：朋友和敌人。我们知道：
>
> - 一个人的朋友的朋友是朋友

- 一个人的敌人的敌人是朋友

现在要对这些人进行组团。两个人在一个团体内当且仅当这两个人是朋友。请求出这些人中最多可能有的团体数。

【输入描述】

第一行输入一个整数 n 代表人数。

第二行输入一个整数 m 表示接下来要列出 m 个关系。

接下来 m 行，每行一个字符 opt 和两个整数 p, q，分别代表关系（朋友或敌人），有关系的两个人之中的第一个人和第二个人。其中 opt 有两种可能：

- 如果 opt 为 `F`，则表明 p 和 q 是朋友。
- 如果 opt 为 `E`，则表明 p 和 q 是敌人。

【输出描述】

一行一个整数代表最多的团体数。

【示例一】

输入：

```
6
4
E 1 4
F 3 5
F 4 6
E 1 2
```

输出：

```
3
```

【解法】

扩展域并查集模板题：

- a 和 b 如果是朋友，那就直接合并在一起；
- a 和 b 如果是敌人，那就把 a 和 $b+n$ 以及 $a+n$ 和 b 合并在一起。

【参考代码】

```cpp
#include <iostream>

using namespace std;
```

```cpp

const int N = 1010;

int n, m;
int fa[N * 2]; // 扩展域并查集

int find(int x)
{
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}

// 一定要让朋友域作为父结点
void un(int x, int y)
{
    fa[find(y)] = find(x);
}

int main()
{
    cin >> n >> m;

    // 初始化
    for(int i = 1; i <= n * 2; i++) fa[i] = i;

    while(m--)
    {
        char op; int x, y;
        cin >> op >> x >> y;
        if(op == 'F')
        {
            un(x, y);
        }
        else // 敌人
        {
            un(x, y + n);
            un(y, x + n);
        }
    }

    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        if(fa[i] == i) ret++;
    }

    cout << ret << endl;

```

```
51        return 0;
52    }
```

3.6.2 食物链

题目来源： 洛谷

题目链接： P2024 [NOI2001] 食物链

难度系数： ★★★

【题目描述】

动物王国中有三类动物 A,B,C，这三类动物的食物链构成了有趣的环形。A 吃 B，B 吃 C，C 吃 A。

现有 N 个动物，以 $1\sim N$ 编号。每个动物都是 A,B,C 中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这 N 个动物所构成的食物链关系进行描述：

- 第一种说法是 `1 X Y`，表示 X 和 Y 是同类。
- 第二种说法是 `2 X Y`，表示 X 吃 Y。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 当前的话与前面的某些真的话冲突，就是假话；
- 当前的话中 X 或 Y 比 N 大，就是假话；
- 当前的话表示 X 吃 X，就是假话。

你的任务是根据给定的 N 和 K 句话，输出假话的总数。

【输入描述】

第一行两个整数，N,K，表示有 N 个动物，K 句话。

第二行开始每行一句话（按照题目要求，见样例）

【输出描述】

一行，一个整数，表示假话的总数。

【示例一】

输入：

```
100 7
1 101 1
```

```
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

输出：

```
3
```

【解法】

针对 x ，扩展三个域：同类域 x，捕食域 x + n，被捕食域 x + n + n。

如果 x 和 y 是同类：

- x 和 y 是同类
- x + n 与 y + n 是同类
- x + n + n 与 y + n + n 是同类

如果 x 捕食 y：

- x + n 与 y 同类
- x 与 y + n + n 同类
- x + n + n 与 y + n 同类

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 5e4 + 10;

int n, k;
int fa[N * 3]; // 扩展域并查集

int find(int x)
{
    return x == fa[x] ? x : fa[x] = find(fa[x]);
```

```cpp
}

void un(int x, int y)
{
    fa[find(x)] = find(y);
}

int main()
{
    cin >> n >> k;

    // 初始化
    for(int i = 1; i <= n * 3; i++) fa[i] = i;

    int ret = 0;
    while(k--)
    {
        int op, x, y; cin >> op >> x >> y;
        if(x > n || y > n) ret++;
        else if(op == 1) // 同类
        {
            // y -> x 以及 x -> y
            if(find(x) == find(y + n) || find(x) == find(y + n + n)) ret++;
            else
            {
                // 维护这个关系
                un(x, y);
                un(x + n, y + n);
                un(x + n + n, y + n + n);
            }
        }
        else // x -> y
        {
            // x 和 y 是同类，y -> x
            if(find(x) == find(y) || find(x) == find(y + n)) ret++;
            else
            {
                // 维护这个关系
                un(x, y + n + n);
                un(y, x + n);
                un(x + n + n, y + n);
            }
        }
    }

    cout << ret << endl;

```

```
60        return 0;
61    }
```

3.7 带权并查集

1. 带权并查集的概念

带权并查集在普通并查集的基础上，为每个结点增加了一个权值。这个权值可以表示当前结点与父结点之间的关系、距离或其他信息（注意，由于我们有路径压缩操作，所以最终这个权值表示的是当前结点相对于根结点的信息）。有了这样一个权值，就可以推断出集合中各个元素之间的相互关系。

2. 带权并查集的实现

我们以最简单的距离问题为例，实现一个能够查询任意两点之间距离的并查集。

实现带权并查集的核心是在进行 `Find` 和 `Union` 操作时，不仅要维护集合的结构，还要维护结点的权值。

注意：带权并查集的实现是多种多样的，基本上换一道题，实现的代码就要更改。因此一定要重点关注实现过程的思考方式，这才是通用的。

初始化 `init`：

```
1   const int N = 1e5 + 10, INF = 0x3f3f3f3f;
2   int n;
3   int fa[N], d[N];
4
5   void init()
6   {
7       for(int i = 1; i <= n; i++)
8       {
9           fa[i] = i;
10          d[i] = 0; // 根据题目要求来初始化
11      }
12  }
```

查询根节点操作 `find`：

```cpp
int find(int x)
{
    if(fa[x] == x) return x;

    int t = find(fa[x]); // 这句代码一定要先执行，先让父结点挂在根节点的后面
    d[x] += d[fa[x]]; // 注意，可能会根据权值的意义有所改变

    return fa[x] = t;
}
```

合并操作 `union`：

```cpp
// x 所在集合与 y 所在集合合并，x 与 y 之间的权值是 w
void un(int x, int y, int w)
{
    int fx = find(x), fy = find(y);
    if(fx != fy) // 不在同一个集合中
    {
        fa[fx] = fy;
        d[fx] = d[y] + w - d[x]; // 注意，可能会根据权值的意义有所改变
    }
}
```

查询距离操作 `query`：

```cpp
// 查询 x 到 y 的距离
int query(int x, int y)
{
    int fx = find(x), fy = find(y);

    if(fx != fy) return INF; // 如果不在同一个集合中，说明距离未知
    return d[y] - d[x];
}
```

3.7.1 食物链

> **题目来源： 洛谷**

题目链接： P2024 [NOI2001] 食物链

难度系数： ★★★

【题目描述】

动物王国中有三类动物 A,B,C，这三类动物的食物链构成了有趣的环形。A 吃 B，B 吃 C，C 吃 A。

现有 N 个动物，以 $1\sim N$ 编号。每个动物都是 A,B,C 中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这 N 个动物所构成的食物链关系进行描述：

- 第一种说法是 `1 X Y`，表示 X 和 Y 是同类。
- 第二种说法是 `2 X Y`，表示 X 吃 Y。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 当前的话与前面的某些真的话冲突，就是假话；
- 当前的话中 X 或 Y 比 N 大，就是假话；
- 当前的话表示 X 吃 X，就是假话。

你的任务是根据给定的 N 和 K 句话，输出假话的总数。

【输入描述】

第一行两个整数，N,K，表示有 N 个动物，K 句话。

第二行开始每行一句话（按照题目要求，见样例）

【输出描述】

一行，一个整数，表示假话的总数。

【示例一】

输入：

```
100 7
1 101 1
2 1 2
2 2 3
2 3 3
1 1 3
2 3 1
1 5 5
```

> 输出：
>
> 3

【解法】

把真话里面的相互关系，用"带权并查集"维护起来，权值表示当前节点相对于根节点的距离。那么对于集合中的任意两点 x 和 y ：

- 如果 $(d[y] - d[x]) \ \% \ 3 == 0$ ，表示两者是同类关系；
- 如果 $(d[y] - d[x]) \ \% \ 3 == 1$ ，表示两者是捕食关系；
- 如果 $(d[y] - d[x]) \ \% \ 3 == 2$ ，表示两者是天敌关系。

`find` 操作：

- 更新 d 数组：按照最基础的距离更新的方式， `d[x] = d[x] + d[fa[x]]` ；

`union` 操作：

- 如果 x 和 y 是同类，那么边权就是 0 ；
- 如果 x 吃 y ，那么边权就是 1 ；

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 5e4 + 10;

int n, k;
int fa[N], d[N]; // 带权并查集

int find(int x)
{
    if(fa[x] == x) return x;

    int t = find(fa[x]);
    d[x] += d[fa[x]];
    return fa[x] = t;
}

void un(int x, int y, int w)
{
```

```cpp
    int fx = find(x), fy = find(y);
    if(fx != fy)
    {
        fa[fx] = fy;
        d[fx] = d[y] + w - d[x];
    }
}

int main()
{
    cin >> n >> k;

    for(int i = 1; i <= n; i++) fa[i] = i;

    int ret = 0;
    while(k--)
    {
        int op, x, y; cin >> op >> x >> y;
        int fx = find(x), fy = find(y);

        if(x > n || y > n) ret++;
        else if(op == 1) // 同类
        {
            if(fx == fy && ((d[y] - d[x]) % 3 + 3) % 3 != 0) ret++;
            else un(x, y, 0);
        }
        else // x -> y
        {
            if(fx == fy && ((d[y] - d[x]) % 3 + 3) % 3 != 1) ret++;
            else un(x, y, 2);
        }
    }

    cout << ret << endl;

    return 0;
}
```

3.7.2 银河英雄传说

> **题目来源：** 洛谷
>
> **题目链接：** P1196 [NOI2002] 银河英雄传说
>
> **难度系数：** ★★★

【题目描述】

杨威利擅长排兵布阵，巧妙运用各种战术屡次以少胜多，难免恣生骄气。在这次决战中，他将巴米利恩星域战场划分成 30000 列，每列依次编号为 1,2,…,30000。之后，他把自己的战舰也依次编号为 1,2,…,30000，让第 i 号战舰处于第 i 列，形成“一字长蛇阵”，诱敌深入。这是初始阵形。当进犯之敌到达时，杨威利会多次发布合并指令，将大部分战舰集中在某几列上，实施密集攻击。合并指令为 `M i j`，含义为第 i号战舰所在的整个战舰队列，作为一个整体（头在前尾在后）接至第 j 号战舰所在的战舰队列的尾部。显然战舰队列是由处于同一列的一个或多个战舰组成的。合并指令的执行结果会使队列增大。

然而，老谋深算的莱因哈特早已在战略上取得了主动。在交战中，他可以通过庞大的情报网络随时监听杨威利的舰队调动指令。

在杨威利发布指令调动舰队的同时，莱因哈特为了及时了解当前杨威利的战舰分布情况，也会发出一些询问指令：`C i j`。该指令意思是，询问电脑，杨威利的第 i 号战舰与第 j 号战舰当前是否在同一列中，如果在同一列中，那么它们之间布置有多少战舰。

作为一个资深的高级程序设计员，你被要求编写程序分析杨威利的指令，以及回答莱因哈特的询问。

【输入描述】

第一行有一个整数 T（1≤T≤5×10^5），表示总共有 T 条指令。

以下有 T 行，每行有一条指令。指令有两种格式：

1. `M i j`：i 和 j 是两个整数（1≤i,j≤30000），表示指令涉及的战舰编号。该指令是莱因哈特窃听到的杨威利发布的舰队调动指令，并且保证第 i 号战舰与第 j 号战舰不在同一列。
2. `C i j`：i 和 j 是两个整数（1≤i,j≤30000），表示指令涉及的战舰编号。该指令是莱因哈特发布的询问指令。

【输出描述】

依次对输入的每一条指令进行分析和处理：

- 如果是杨威利发布的舰队调动指令，则表示舰队排列发生了变化，你的程序要注意到这一点，但是不要输出任何信息。
- 如果是莱因哈特发布的询问指令，你的程序要输出一行，仅包含一个整数，表示在同一列上，第 i 号战舰与第 j 号战舰之间布置的战舰数目。如果第 i 号战舰与第 j 号战舰当前不在同一列上，则输出 −1。

【示例一】

输入：

```
4
M 2 3
C 1 2
M 2 4
```

> C 4 2
>
> 输出：
>
> -1
>
> 1

【解法】

这道题中有明显的边权关系，因此可以用"带权并查集"解决。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 3e4 + 10;

int n = 3e4;
int fa[N], d[N], cnt[N]; // 维护集合的合并、维护权值、维护集合的大小

int find(int x)
{
    if(fa[x] == x) return x;

    int t = find(fa[x]);
    d[x] += d[fa[x]];
    return fa[x] = t;
}

void un(int x, int y)
{
    int fx = find(x), fy = find(y);

    if(fx != fy)
    {
        fa[fx] = fy;
        d[fx] = cnt[fy];
        cnt[fy] += cnt[fx];
    }
}

int query(int x, int y)
{
    int fx = find(x), fy = find(y);
```

```
34
35      if(fx != fy) return -1;
36      else return abs(d[y] - d[x]) - 1;
37  }
38
39  int main()
40  {
41      // 初始化
42      for(int i = 1; i <= n; i++)
43      {
44          fa[i] = i;
45          cnt[i] = 1;
46      }
47
48      int T; cin >> T;
49
50      while(T--)
51      {
52          char op; int x, y;
53          cin >> op >> x >> y;
54          if(op == 'M') // 合并
55          {
56              un(x, y);
57          }
58          else // 查询
59          {
60              cout << query(x, y) << endl;
61          }
62      }
63
64      return 0;
65  }
```

4. 字符串哈希

1. 回忆：哈希函数与哈希冲突

- **哈希函数：**将关键字映射成对应的地址的函数，记为 `Hash(key) = Addr`。
- **哈希冲突：**哈希函数可能会把两个或两个以上的不同关键字映射到同一地址，这种情况称为哈希冲突。

2. 字符串哈希